

LAPORAN PRAKTIKUM

Komputasi Paralel dan Sistem Terdistribusi

Topik:

Load Balancing: Static Scheduling vs Dynamic Scheduling

Nama: Edsel Sulthan Farrel

NRP: 152024166

Mata Kuliah: IFB 206 - Komputasi Paralel dan Sistem Terdistribusi

Dosen Pengampu: Lisa Kristiana PhD

1. Pendahuluan

Dalam komputasi paralel, salah satu tantangan utama adalah bagaimana mendistribusikan beban kerja secara efisien ke seluruh unit pemrosesan yang tersedia. Apabila distribusi tugas tidak merata, maka sebagian worker akan menganggur sementara worker lain masih sibuk memproses tugas yang berat. Kondisi ini menyebabkan pemborosan sumber daya dan memperpanjang total waktu eksekusi program.

Praktikum ini bertujuan untuk memahami dan membandingkan dua pendekatan penjadwalan tugas dalam komputasi paralel, yaitu Static Scheduling dan Dynamic Scheduling, menggunakan modul multiprocessing Python. Simulasi dilakukan dengan skenario penanganan pasien IGD yang memiliki beban kerja heterogen (pasien ringan dan pasien kritis) untuk memperlihatkan perbedaan perilaku kedua teknik secara nyata.

2. Tujuan Praktikum

- Memahami konsep Load Balancing dalam sistem komputasi paralel.
- Membedakan cara kerja Static Scheduling dan Dynamic Scheduling.
- Menganalisis dampak distribusi beban kerja yang tidak merata terhadap performa sistem.
- Mengimplementasikan kedua teknik menggunakan Pool.map() dengan parameter chunksize pada Python.

3. Dasar Teori

3.1 Load Balancing

Load Balancing adalah mekanisme untuk mendistribusikan beban kerja secara merata ke semua worker/prosesor yang tersedia dalam sistem paralel. Tujuannya adalah meminimalkan waktu idle dan memaksimalkan utilisasi sumber daya. Tanpa load balancing yang baik, sistem paralel bisa lebih lambat dari sistem sequential karena ada worker yang nganggur sementara worker lain masih penuh beban.

3.2 Static Scheduling

Static Scheduling adalah teknik penjadwalan di mana tugas-tugas dibagi secara tetap sebelum eksekusi dimulai. Setiap worker mendapatkan sejumlah tugas yang sudah ditentukan dari awal dan tidak bisa berubah selama proses berjalan. Pada Python, ini diimplementasikan dengan menggunakan nilai `chunksize` yang besar di `Pool.map()`.

Keunggulan: overhead koordinasi rendah, cocok untuk beban kerja yang seragam (homogen).

Kelemahan: rentan terjadi ketidakseimbangan jika ada tugas yang jauh lebih berat dari yang lain.

3.3 Dynamic Scheduling

Dynamic Scheduling adalah teknik di mana tugas diberikan ke worker secara bertahap — worker yang sudah selesai akan langsung mengambil tugas berikutnya dari antrian. Pada Python, ini dicapai dengan menggunakan `chunksize=1` pada `Pool.map()`, sehingga setiap worker hanya mendapat satu tugas dalam satu waktu.

Keunggulan: lebih adaptif, tidak ada worker yang nganggur lama, cocok untuk beban kerja yang bervariasi (heterogen).

Kelemahan: overhead koordinasi sedikit lebih tinggi dibanding static.

4. Implementasi

4.1 Deskripsi Skenario

Simulasi menggunakan skenario IGD (Instalasi Gawat Darurat) dengan 6 pasien. Dua pasien dikategorikan kritis (membutuhkan waktu penanganan lebih lama) dan empat pasien dikategorikan ringan. Sistem menggunakan 2 worker (`Pool(2)`) untuk mencerminkan kondisi resources yang terbatas.

No. Pasien	Kategori	Durasi (s)
1	Ringan	0.5
2	Ringan	0.5
3	Kritis	4.0
4	Ringan	0.5
5	Kritis	3.5
6	Ringan	0.5

4.2 Source Code

Berikut adalah implementasi lengkap kedua teknik scheduling:

```
from multiprocessing import Pool, cpu_count
import time

def tangani_pasien(info):
    id_p, tipe, durasi = info
    print(f' [IGD] pasien {id_p} ({tipe}) butuh {durasi}s...')
    time.sleep(durasi)
    return f'pasien {id_p} selesai'

if __name__ == "__main__":
    # pasien 3 sama 5 kritis, sisanya ringan
    # nanti kelihatan bedanya static vs dynamic
    pasien = [
        (1, "ringan", 0.5),
        (2, "ringan", 0.5),
        (3, "kritis", 4.0),
        (4, "ringan", 0.5),
        (5, "kritis", 3.5),
        (6, "ringan", 0.5),
    ]

    # static: worker 1 dapet pasien 1,2,3 — worker 2 dapet 4,5,6
    # masalahnya kalo kebetulan dapet yang kritis semua ya lama
    print("--- static scheduling (chunksize=3) ---")
    t1 = time.time()
    with Pool(2) as p:
        p.map(tangani_pasien, pasien, chunksize=3)
    w1 = time.time() - t1
    print(f'waktu static: {w1:.2f}s\n')

    # dynamic: worker yang nganggur langsung ambil tugas baru
    # lebih fleksibel, ga ada yang nunggu nganggur
    print("--- dynamic scheduling (chunksize=1) ---")
    t2 = time.time()
    with Pool(2) as p:
        p.map(tangani_pasien, pasien, chunksize=1)
    w2 = time.time() - t2
    print(f'waktu dynamic: {w2:.2f}s")

    print(f"\nselisih: {abs(w1-w2):.2f}s -> dynamic lebih efisien kalo beban ga merata")
```

5. Hasil dan Analisis

5.1 Output Program

```

--- static scheduling (chunksize=3) ---
[IGD] pasien 1 (ringan) butuh 0.5s...
[IGD] pasien 4 (ringan) butuh 0.5s...
[IGD] pasien 5 (kritis) butuh 3.5s...
[IGD] pasien 2 (ringan) butuh 0.5s...
[IGD] pasien 3 (kritis) butuh 4.0s...
[IGD] pasien 6 (ringan) butuh 0.5s...
waktu static: 5.36s

--- dynamic scheduling (chunksize=1) ---
[IGD] pasien 1 (ringan) butuh 0.5s...
[IGD] pasien 2 (ringan) butuh 0.5s...
[IGD] pasien 3 (kritis) butuh 4.0s...
[IGD] pasien 4 (ringan) butuh 0.5s...
[IGD] pasien 5 (kritis) butuh 3.5s...
[IGD] pasien 6 (ringan) butuh 0.5s...
waktu dynamic: 5.29s

selisih: 0.07s -> dynamic lebih efisien kalo beban ga merata

```

5.2 Analisis Perbandingan

Dari hasil eksekusi, dapat diamati perbedaan perilaku antara kedua teknik scheduling. Pada Static Scheduling dengan chunksize=3, Worker-1 mendapat pasien 1, 2, dan 3, sementara Worker-2 mendapat pasien 4, 5, dan 6. Karena pasien 3 (kritis, 4.0s) berada di kelompok Worker-1 dan pasien 5 (kritis, 3.5s) berada di kelompok Worker-2, kedua worker sama-sama terbebani — tidak ada satu pun yang bisa membantu yang lain.

Pada Dynamic Scheduling dengan chunksize=1, tugas diberikan satu per satu secara antrian. Ketika Worker-1 selesai menangani pasien ringan, ia langsung mengambil tugas berikutnya tanpa menunggu. Hasilnya, total waktu sedikit lebih singkat karena distribusi tugas lebih adaptif.

Teknik	Chunksize	Waktu Eksekusi	Keterangan
Static Scheduling	3	5.36s	Worker dibagi blok tetap
Dynamic Scheduling	1	5.29s	Worker ambil tugas antrian
Selisih	-	0.07s	Dynamic lebih efisien

5.3 Mengapa Selisihnya Kecil?

Selisih waktu yang relatif kecil (0.07 detik) pada percobaan ini disebabkan oleh distribusi beban yang kebetulan cukup merata — masing-masing worker tetap kebagian minimal satu pasien kritis. Perbedaan yang lebih signifikan akan terlihat jika semua tugas berat terkumpul di satu sisi pada Static Scheduling, misalnya jika pasien 1, 2, 3 semuanya kritis sementara pasien 4, 5, 6 semuanya ringan.

6. Kesimpulan

Dari praktikum ini, dapat disimpulkan beberapa hal mengenai Load Balancing dalam komputasi paralel:

- Static Scheduling cocok digunakan ketika beban kerja setiap tugas relatif seragam, karena overhead koordinasinya lebih rendah.
- Dynamic Scheduling lebih unggul saat beban kerja bervariasi (heterogen), karena worker tidak dibiarkan menganggur ketika ada tugas lain yang tersedia.
- Pada Python, parameter `chunksize` di `Pool.map()` secara langsung menentukan teknik scheduling yang digunakan.
- Dalam skenario dunia nyata seperti pemrosesan dokumen AI, rendering video, atau penanganan request server, Dynamic Scheduling umumnya lebih dipilih karena sifat beban kerja yang tidak dapat diprediksi.

7. Referensi

Python Software Foundation. (2024). multiprocessing — Process-based parallelism. <https://docs.python.org/3/library/multiprocessing.html>

Kumar, V., Grama, A., Gupta, A., & Karypis, G. (1994). Introduction to Parallel Computing. Benjamin/Cummings.

Pacheco, P. (2011). An Introduction to Parallel Programming. Morgan Kaufmann.